

Speed-Based Detection of Objects Moving Away Using a Horizontally Oriented Ultrasonic Sensor

Information Technology Course
Module Machine Learning
Prof. Dr. Andreas Pech

Saquib Attar
1567041
saquib.attar@stud.fra-uas.de

Muhammad Ahsan Ijaz
1566312
muhammad.ijaz@stud.fra-uas.de

Manoj Hanumanthu
1566325
manoj.hanumanthu@stud.fra-uas.de

Aman Basha Patel
1565430
aman.patel@stud.fra-uas.de

Abstract—In many automation and robotics applications, it is important to know not just whether an object is present but also whether it is moving away and how fast. This project explores whether a single ultrasonic sensor mounted horizontally can reliably detect away movement and estimate the speed of the object. Distance is measured using the time-of-flight principle at 20 Hz. Since the raw readings are noisy and often contain echo dropouts during motion, median filtering and a sliding window approach are used to clean the signal and find stable reference points. The speed of the object is then calculated from the change in distance over time using a linear model. A positive slope means the object moved away. To go a step further, we extracted thirteen features from each recording and trained four classifiers which are Decision Tree, Random Forest, KNN, and SVM to distinguish between medium, fast, and diagonal motion. With only 25 recordings available, we used Leave-One-Out Cross-Validation to make the most of the data. SVM gave the best result at 88% accuracy with an F1-score of 0.879, and every single recording was correctly identified as away movement. The average MAE and RMSE of the regression fit were 0.408 m and 0.481 m respectively. Overall, the results suggest that even a simple and inexpensive setup can work well for this kind of detection task when paired with the right processing and learning pipeline.

Keywords—Ultrasonic sensor, time-of-flight, object detection, speed estimation, away movement, signal processing, median filtering, machine learning, classification, SVM

I. INTRODUCTION

Ultrasonic sensors are now commonly used in industrial automation, robotics, surveillance, and safety systems. Their widespread adoption stems from their affordability, physical durability, and independence from ambient light. Ultrasonic sensors, in contrast to cameras or infrared sensors, maintain reliable functionality in environments characterized by dust, fog, or darkness, rendering them particularly suitable for applications in factory settings, warehouses, and outdoor surveillance. Utilizing the time-of-flight principle, these devices emit

a short ultrasonic pulse, which then reflects off a nearby object; the sensor subsequently calculates the duration of the echo's return. The distance to the object is then determined by calculating the time delay, given the known speed of sound.

Ultrasonic sensors are typically utilized in conventional configurations to ascertain the presence of an object within a specified range or to detect its approach towards the sensor. These tasks are relatively uncomplicated, as the echo signal's strength, consistency, and trackability improve as an object approaches. Conversely, detecting objects receding from the sensor presents a fundamentally distinct challenge. As the object recedes, the echo weakens, the signal becomes noisier, and in many cases the sensor loses the echo entirely for several frames before picking it up again at a different position. These echo dropouts make it difficult to track the object continuously and to estimate its speed from the raw distance readings. Despite this difficulty, away movement detection is important in several practical contexts. On a conveyor belt, for instance, knowing the speed at which items move away from a sensor station can help in quality control and throughput monitoring. In the context of warehouse robotics, determining the departure of a package or an individual from a designated area is advantageous for both collision avoidance and path planning. Similarly, in perimeter security applications, the capacity to ascertain the direction and velocity of a moving object aids in differentiating between normal activities and potential security breaches.

Several studies have examined how ultrasonic sensors can be used to detect motion and estimate speed. Thong-un and Wongsaraj [1] formulated a technique for tracking the position and velocity of moving objects within a three-dimensional space, employing ultrasonic transducers. Their methodology illustrated that ultrasonic signals possess sufficient data to reconstruct motion trajectories; however, their system neces-

sitated the deployment of multiple sensors configured in a particular geometric arrangement. Alonso et al. [2] studied how ultrasonic sensors could be used in city traffic, specifically looking at how to estimate vehicle speed using time-of-flight data from sensors placed along the road. Their study confirmed that ultrasonic speed estimation works well in real-world outdoor settings. However, it required a setup with multiple sensors, which was designed for specific traffic lanes. Bowler et al. [3] took a different approach by combining ultrasonic sensing with machine learning techniques to monitor industrial processes. The study showed that using raw ultrasonic signals with learning algorithms allows for the extraction of useful patterns, which goes beyond just measuring distance. These studies, taken together, show that estimating speed using ultrasound is possible. However, most current methods either require many sensors, depend on controlled lab conditions, or are only suitable for situations where the object is moving toward the sensor, not away from it.

What remains relatively underexplored is a simple, single-sensor approach that can detect away movement and also classify the speed of the object into meaningful categories. This paper addresses that gap. The proposed system uses one horizontally mounted ultrasonic sensor, applies signal processing to handle noise and echo dropouts, estimates speed using a linear regression model, and classifies the motion type using machine learning. The aim is to show that even with a minimal hardware setup and a lightweight processing pipeline, it is possible to achieve reliable away movement detection and speed-based classification.

II. METHODOLOGY

This project uses a structured pipeline that starts with raw ultrasonic echo data and ends with a machine learning-based classification of the type of motion. There are different steps in the overall process. First, the sensor hardware picks up the raw echo signal. The echo data is then changed into distance measurements using the time-of-flight principle. Filtering methods are used to clean up the distance signal that comes out, which makes it less noisy. Using a linear regression model, the speed of the object is estimated from the cleaned signal, and the direction of motion is found. Finally, a set of numbers is taken from each recording and used to train machine learning classifiers that can tell the difference between different kinds of object movement. The following subsections go into great detail about each of these steps.

A. Ultrasonic Sensor and Red Pitaya Measurement Board

The SRF02 ultrasonic range finder [4] is a small, single-transducer sensor that is often used to measure short distances. The SRF02 is different from dual-transducer sensors because it only has one transducer that sends and receives the ultrasonic pulse. This makes the sensor smaller and easier to use, but it also means that the minimum distance that can be measured is about 15 cm. This is because the transducer needs time to switch from sending to receiving mode before it can pick up the echo. The sensor works well with a 5V power source.

The sensor is connected to a Red Pitaya STEMLAB measurement board [5], which collects the data. Xilinx made the Red Pitaya, which is a flexible open-source device that uses a System-on-a-Chip (SoC) architecture. Depending on how it is set up, it can work as an oscilloscope, spectrum analyzer, LCR meter, or signal generator. For this project, it is used to record the raw analog echo signal that the ultrasonic sensor sends back. The board samples the echo waveform at a rate of 1,953,125 Hz, which gives a clear picture of the pulse that is coming back. The GNU/Linux operating system runs the Red Pitaya platform. It lets data be sent wirelessly to a laptop, where it can be analyzed and processed later.

The main part of Red Pitaya has 16 standard input and output ports, as well as two analog RF inputs and outputs. The board has a micro-SD card slot for storage, an RJ45 socket for Ethernet connections, a USB port for connecting peripherals, and a micro-USB port for console access.

The sensor was placed horizontally on a flat surface, pointing in the direction that things would move away from. This horizontal orientation is important because it means that the ultrasonic beam is aimed along the object's path of motion. This lets the sensor keep track of changes in distance as the object moves away. Figure 1 shows the whole hardware setup, which includes the ultrasonic sensor that is connected to the Red Pitaya board.

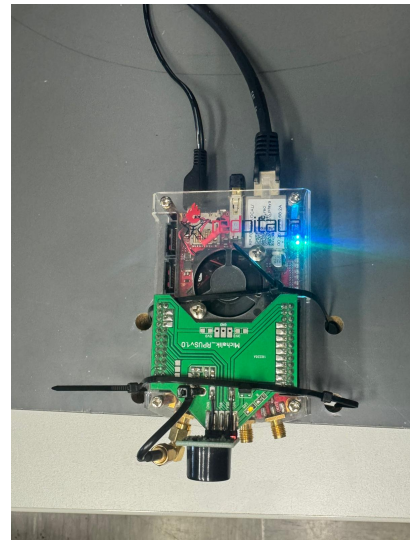


Fig. 1. Red Pitaya with Ultrasonic sensor

B. Time-of-Flight Distance Conversion

In each frame, the raw echo waveform has thousands of amplitude samples. When the ultrasonic pulse hits something and bounces back, it makes the received signal drop sharply. To find the position of this dip, you need to find the sample in the waveform with the lowest amplitude value. This gives the peak index, which shows how many samples came after the pulse was sent and before the echo came.

The time-of-flight principle is used to turn this peak index into a physical distance. To figure out the speed of sound, you first use the formula that depends on the temperature:

$$c = 331.3 + 0.606 \cdot T \quad (1)$$

T is the temperature of the air in degrees Celsius. At 20 degrees, this means a speed of about 343.4 meters per second. Then, the distance to the object in one direction is calculated as:

$$d = \frac{n_{\text{peak}} \cdot c}{2 \cdot f_s} \quad (2)$$

The echo sampling frequency, f_s , is 1,953,125 Hz. The division by 2 is because the ultrasonic pulse goes to the object and back, so the real distance is half of the total path length. This change is made to every frame in the recording, which gives us a time series of distance values sampled at 20 Hz.

C. Signal Filtering and Echo Dropout Handling

The time-of-flight conversion gives a raw distance signal that has a lot of different kinds of noise. Frames can display sudden increases, decreases, or subtle shifts. These variations are caused by sensor instability, reflections in the environment, or slight changes in the echo's shape.

A median filter with a kernel size of 5 is used to get rid of this noise [6]. The median filter was chosen instead of a simple moving average because it is better at dealing with outliers. If one frame had a sudden spike, a moving average would be pulled toward that extreme value. The median filter, on the other hand, just picks the middle value from the window of 5 consecutive frames and ignores outliers.

The signal can still have bad areas even after filtering. One of the biggest problems encountered in this project was echo dropout. There are times when the sensor completely loses the echo when an object is being pushed away. This happens because the object is moving, and the reflected pulse may not come back strongly enough for the sensor to pick it up during the transition. When there is a dropout, the distance reading goes down to almost zero, which is not where the object really is. Fig. 2 shows an example of the filtered distance signal that shows both the clean motion and the echo dropout behavior.

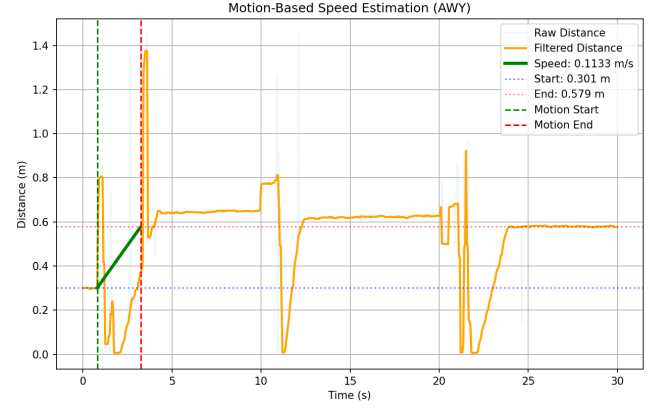


Fig. 2. Example distance-time plot for a medium speed recording showing raw signal, filtered signal, regression line, and echo dropout behavior.

Another issue that was observed is what is referred to as a wall-read scenario. In some recordings, the object was pushed so far or so quickly that it went completely out of the sensor's range of detection. When the object stops bouncing back the ultrasonic pulse, the sensor picks up whatever is in front of it, which is usually a wall or a surface behind where the object used to be. These recordings were recognized and marked as not trustworthy.

The sliding window technique is employed to ascertain the object's point of origin and final location. To determine the starting position, the algorithm sequentially analyzes the filtered distance signal, beginning from the initial point, to identify the first continuous sequence of frames displaying uniform readings. Stability is signified by a standard deviation of less than 0.02 m within the specified window.

The algorithm sequentially evaluates window sizes of 5, 8, 10, and 15 frames, searching for a stable segment. The median value of this identified section serves as the basis for determining the starting distance. Subsequently, the algorithm analyzes the final 30% of the recording to ascertain the terminal position. It calculates the median, eliminates any frames deviating by more than 0.3 m from this median, and then computes the median of the remaining, filtered frames to establish the end distance.

D. Speed Estimation and AWY Detection

Once the starting and ending positions have been identified, the speed of the object can be estimated. The method uses a linear regression model that assumes the distance $d(t)$ changes linearly with time during the motion period:

$$d(t) = d_{\text{start}} + v \cdot (t - t_{\text{start}}) \quad (3)$$

The starting position is d_{start} , the time when motion starts is t_{start} , and the object's speed is v . The slope v is found by using the two stable reference points:

$$v = \frac{d_{\text{end}} - d_{\text{start}}}{t_{\text{end}} - t_{\text{start}}} \quad (4)$$

where d_{end} is the last stable position and t_{end} is the time when the object gets to its last position.

Two checks are done to find out exactly where the motion period starts and ends. The motion start is the first frame where the filtered distance is more than 0.05 m away from the starting value. To find the motion end, the algorithm scans forward from the motion start and looks for a stretch of 20 consecutive frames where the median is within 0.1 m of the final value and at least 70% of the frames in that window are within 0.2 m of the final value.

The detection of away movement is straightforward. If the estimated speed is positive, it means that the object's distance from the sensor grew over time, which confirms that the object moved away from the sensor.

Two standard metrics are used to see how well the linear regression model fits the real signal during the motion period. The Mean Absolute Error (MAE) tells how far off the regression line is from the actual filtered distance on average:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |d_{\text{actual},i} - d_{\text{predicted},i}| \quad (5)$$

The Root Mean Square Error (RMSE) puts more weight on bigger differences:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (d_{\text{actual},i} - d_{\text{predicted},i})^2} \quad (6)$$

If the MAE and RMSE are low, it means that the object moved smoothly and the signal closely followed the expected linear path. A high MAE and RMSE usually mean that there were echo dropouts or other problems during the motion period. Figure 3 shows an example of the regression line fitted over the motion period, with the start and end reference points marked.

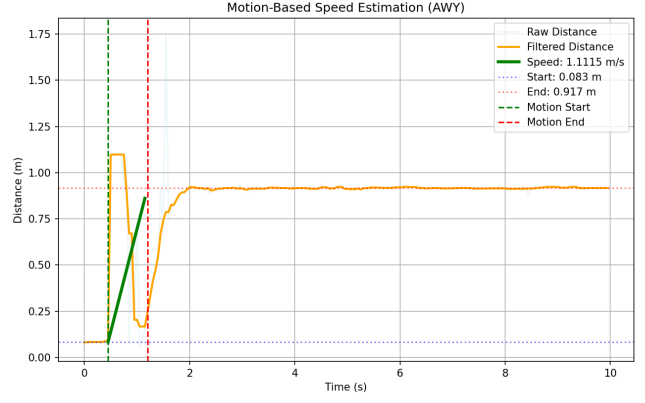


Fig. 3. Example distance-time plot for a fast speed recording showing raw signal, filtered signal, regression line, and echo dropout behavior.

E. Feature Extraction

There are about 200 frames of raw echo data in each recording in the dataset, as well as the derived distance measurements, filtered signals, and computed speed values. But machine learning classifiers cannot work with raw waveforms or time series that are different lengths. They need a set number of numbers for each sample. So, a set of 13 features was taken from each recording. This compressed the whole signal into one row of numbers that shows the most important parts of that experiment.

The features were picked so that they would show three different things about the signal. The first group talks about the motion's spatial properties, like where the object started, where it ended, how far it went, and how fast it went. The second group talks about how echo dropout works, which is very different for different types of motion. The third group talks about the echo signal's quality and other features. Table I has the full list of features.

TABLE I
DESCRIPTION OF THE 13 EXTRACTED FEATURES

Feature	Description
Start Distance	Stable distance reading before motion
End Distance	Stable distance reading after motion
Displacement	Difference between end and start distance
Duration	Time between motion start and motion end
Regression Speed	Slope of linear model: displacement / duration
Number of Dropouts	Count of frames with distance below 0.05 m
Dropout Ratio	Number of dropouts divided by total frames
Mean Amplitude	Average peak echo amplitude across all frames
Std Amplitude	Standard deviation of peak echo amplitude
Mean Echo Energy	Average std. deviation of echo waveform per frame
Std Echo Energy	Variation of echo energy across frames
Signal Range	Maximum minus minimum of filtered distance
Start Stability	Standard deviation of first 10 filtered frames

These features were not picked at random. It was found during the data exploration phase that diagonal recordings always had longer start and end distances than medium and fast recordings. This is because the object was farther from

the sensor when measured at an angle. The object moved slowly enough to stay in the sensor's blind zone, which caused medium recordings to have more echo dropouts. When recorded quickly, the motion lasted less time and moved faster. The features are meant to show these natural differences between the classes.

F. Confusion Matrix

A confusion matrix is a table that is commonly used in machine learning to check how well a classification model works [7]. It works by looking at the actual class labels of the test data and the predicted class labels of the model. For a problem with more than one class, like ours, the confusion matrix is a 3×3 table where the rows show the real classes and the columns show the predicted classes. Each cell shows how many samples from a real class were put into a predicted class. The cells that are on the diagonal show correct predictions, and the cells that are off-diagonal show wrong predictions. Figure 4 shows the basic structure of a confusion matrix.

		<u>True class</u>	
		p	n
<u>Hypothesized class</u>	Y	True Positives	False Positives
	N	False Negatives	True Negatives

Fig. 4. Confusion matrix

Several metrics can be derived from the confusion matrix. The ratio of correct predictions to total predictions is called accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (7)$$

Precision tells how many of the predicted positives were right:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (8)$$

Recall tells how many of the real positives were found correctly:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (9)$$

F1-Score is the harmonic mean of precision and recall, which gives a balanced score:

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (10)$$

A score of 1.0 on the *F1* scale means perfect classification, while a score close to 0 means the model is not working well. For this project, these metrics are calculated for each motion class and then averaged across all classes.

G. Decision Tree

One of the easiest to understand classification algorithms is a Decision Tree [8]. It learns a set of if-then rules from the training data, which is arranged in a tree-like way. The algorithm starts at the root and picks the feature and threshold that best split the data into groups. Each group should ideally only have samples from one class. This splitting process happens again at each branch until the tree is deep enough or the groups are pure enough.

The tree might learn a rule like "if end_distance is greater than 2.0 meters, classify as Diagonal." It might split again at the next level, depending on speed or dropout count. Decision Trees are very easy to understand because the rules come entirely from the training data and can be understood by a person.

Decision Trees' biggest problem is that they tend to overfit. If the tree gets too deep, it might remember the exact patterns in the training data, even the noise, instead of learning the general patterns. In this project, the maximum depth was set to 3 levels to stop this from happening. Figure 5 shows a general picture of how a Decision Tree divides data.

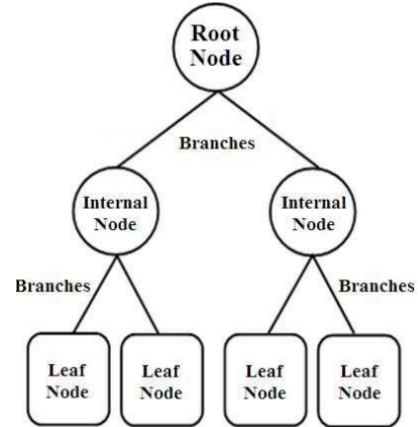


Fig. 5. Decision Tree structure [15]

A Decision Tree classifier works by going through the following steps:

1. Data Preparation: The dataset is set up so that the extracted features are in the input columns and the motion class labels are in the target variable. `StandardScaler` is used to scale the features so that their ranges are all the same.

2. Making the Tree: The algorithm looks at each feature and tries every possible threshold value to find the split that makes the child groups the most pure. The *Gini impurity* criterion is often used to measure purity. It tells how often a random sample from a group would be misclassified if it were given a random label from the group's distribution.

3. Recursive Splitting: After finding the best split at the root, the same thing is done for each child group. This keeps happening over and over until the maximum depth is reached or a group only has samples from one class.

4. Prediction: The algorithm starts at the root and follows the branch that matches the feature values of the new sample at each split to classify it. The prediction is the class label that is given to the leaf node when it gets there.

5. Evaluation: The confusion matrix and derived metrics like accuracy, precision, recall, and *F1-score* are used to compare the model's predictions to the actual labels.

H. Random Forest

Random Forest [9] is an ensemble learning method that builds on the Decision Tree method. Random Forest builds 100 trees instead of just one. Each tree is trained on a different random subset of the data and features. When a tree makes a prediction, it votes, and the class that gets the most votes is chosen as the final output.

This method has a number of benefits over just one Decision Tree. Random Forest is naturally resistant to overfitting because no one tree can memorize a pattern and take over the ensemble. This is because each tree only sees a part of the data and features. Another good thing about Random Forest is that it gives a list of features in order of importance by keeping track of how much each one affects the quality of splits across all trees. This was used in this project to figure out which signal features are best at telling different types of motion apart. Figure 6 shows how a Random Forest classifier with majority voting works.

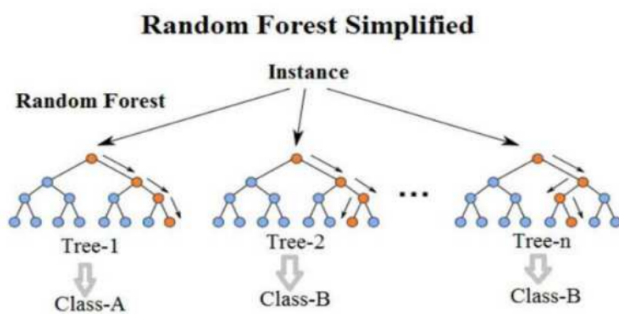


Fig. 6. Random Forest [14]

Here are the steps that a Random Forest classifier goes through to do its job:

1. Preparing the Data: The feature matrix and class labels are set up the same way they are for the Decision Tree. `StandardScaler` is used to make the feature ranges more even.

2. Bootstrap Sampling: A random sample of the training data is taken with replacement for each tree in the forest. This means that some training samples may show up more than once in a tree's training set, while others may not show up at all. This method is called *bagging*.

3. Constructing Individual Trees: The standard procedure for creating Decision Trees is used, but with a key difference. At each split, only a randomly chosen subset of the available features is considered. This randomness helps to create more diverse trees, which prevents them from all learning the same patterns.

4. Combining Predictions: Every tree in the forest looks at a new sample to see what it is. Every tree makes its own prediction on its own. Majority voting decides the final class. The class that gets the most votes from all 100 trees is the one that wins.

5. Calculation of Feature Importance: After training, the algorithm figures out how much each feature helped lower impurity across all splits in all trees. Features that frequently produce clear divisions and significant information gain are particularly important.

6. Evaluation: The ensemble's predictions are assessed using the same metrics as the individual models, including the confusion matrix, accuracy, precision, recall, and the *F1-score*.

I. K-Nearest Neighbors

K-Nearest Neighbors, or KNN [10], classifies things in a different way. KNN does not learn any clear rules while it is training, unlike tree-based methods. It just saves all the training samples instead. The algorithm finds the *k* closest training samples to a new sample and gives it the class that is most common among those neighbors.

For this project, *k* was set to 3, and *Euclidean distance* was used to find out how close samples were to each other in the 13-dimensional feature space. Feature scaling is very important for KNN because if one feature has values in the thousands and another has values between 0 and 0.15, the distance calculation will be based on the larger feature, even if it is not that important. The normalization done by the `StandardScaler` makes sure that all features have the same effect.

KNN is easy to use and often works, but it depends on the choice of *k* and features that do not matter. With 25 samples, *k* = 3 strikes a good balance between being too sensitive to

individual outliers and being too general. Figure 7 shows how KNN classification works.



Fig. 7. K-Nearest Neighbors classification

The following steps are part of how a KNN classifier works:

1. **Preparing the data:** The training data and its class labels are stored in memory. `StandardScaler` is used to make sure that all the features are on the same scale, which is important for making sense of distance calculations.
2. **Picking K:** The value of k tells how many neighbors to consult for each prediction. If k is set to a small number, the model will be sensitive to changes and noise in the area. If k is set to a large number, the model will be smoother, but it may make the lines between classes less clear. This project set k to 3 because the dataset was small.
3. **Distance Calculation:** When a new sample arrives, the *Euclidean distance* between the new sample and every stored training sample is calculated. The distance within a 13-dimensional feature space is determined by taking the square root of the sum of the squared differences for each feature.
4. **Identifying Nearest Neighbors:** The training samples are ordered according to their proximity to the new sample. Subsequently, the k samples exhibiting the least distance to the new sample are chosen. These nearest samples constitute the neighbors that will contribute to the prediction concerning the new sample.
5. **Making Predictions:** The class labels of the k nearest neighbors are examined, and the class that appears most often is assigned to the new sample. If there is a tie, the class of the single nearest neighbor is typically used to break the tie.
6. **Evaluation:** The predictions are compared against the actual labels using the confusion matrix and associated metrics.

J. Support Vector Machine

Support Vector Machine (SVM) [11] works by finding the best separating line, or hyperplane, that divides different classes

in the feature space. It looks for a hyperplane that makes the biggest gap between the classes. Support vectors are the samples that are closest to this boundary. They are the data points that show where the boundary is.

In a lot of real-life situations, a straight line cannot separate the classes. SVM uses a kernel function to project the data into a higher-dimensional space where a separating boundary can be found. A Radial Basis Function (RBF) kernel was used for this project. It makes a decision boundary that is curved and flexible.

SVM works well on small datasets because it only looks at the support vectors near the class boundaries instead of trying to model the whole data distribution. In this project, the overlap between the Fast and Medium classes is where classification is hardest, and SVM is a good fit for this problem because it focuses on the boundary region. Figure 8 shows how SVM works.

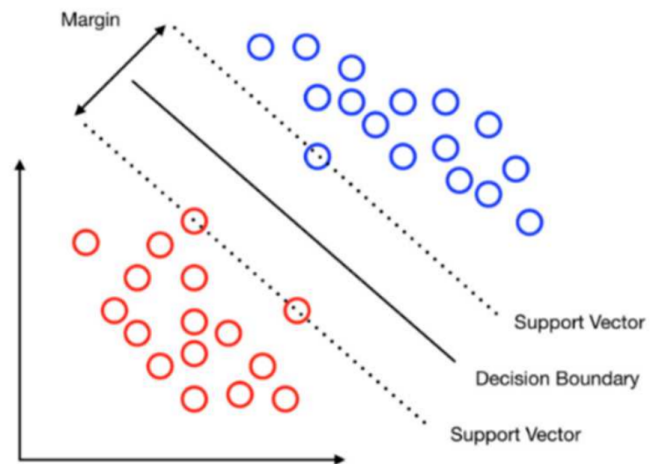


Fig. 8. Support Vector Machine [14]

The steps that an SVM classifier takes to work are as follows:

1. **Data Preparation:** The features are scaled using `StandardScaler`. This is especially important for Support Vector Machines (SVMs) because the algorithm's performance depends on calculating distances in the feature space. If the features aren't scaled and have large numerical ranges, they will dominate the placement of the decision boundary.
2. **Kernel Selection:** The kernel function decides how to change the data before finding the line that separates the two groups. The RBF kernel was selected due to its capability to manage non-linear class boundaries by transforming the original features into a higher-dimensional space, facilitating linear separation.

3. Finding the Best Hyperplane: The algorithm looks for the hyperplane that makes the space between the closest samples of different classes as big as possible. The margin is the space between the hyperplane and the closest data point on either side. A wider margin usually means that the model will do a better job of generalizing to new data.

4. Finding Support Vectors: Support vectors are the training samples that are exactly on the margin boundary or inside the margin. These are the most important pieces of information that tell where the decision boundary is and how it is oriented. The final model is not affected by any of the other training samples.

5. Making Predictions: The algorithm figures out which side of the decision boundary a new sample is on. In a project with three classes, the SVM uses a *one-versus-one* strategy, which means that it learns separate boundaries for each pair of classes and then makes the final prediction by voting across all pairwise classifiers.

6. Evaluation: The confusion matrix, accuracy, precision, recall, and $F1$ -score are used to check how well the model's predictions match reality.

K. Leave-One-Out Cross-Validation

When evaluating a machine learning model, it is crucial to use data that was not part of the training set [12]. The most common way to do this is to split the data into two parts: one for training and one for testing. Usually, 80% of the data is used for training and 20% for testing. But since this project only has 25 recordings, a 20% test split would only leave 5 samples for testing, which is not enough to make reliable conclusions.

To get around this problem, Leave-One-Out Cross-Validation (LOO-CV) was used. In this method, the model is trained with 24 samples and then tested with the 1 sample that is left over. This happens 25 times, once for each sample. After all the rounds, each sample has been tested once, and the results are put together to find the accuracy, confusion matrix, precision, recall, and $F1$ -score.

The best thing about LOO-CV is that it uses 24 out of 25 samples for training each time, which means that each round has the most training data possible. The cost of training the model 25 times is very low because the dataset is small and the models are simple. Figure 9 shows how the LOO-CV process works.

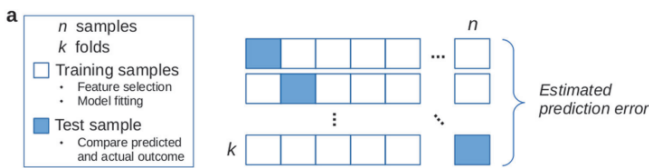


Fig. 9. Leave-One-Out Cross-Validation procedure with 25 recordings. Each round uses 24 samples for training and 1 for testing, repeated 25 times until every sample has been the test sample exactly once [13].

III. IMPLEMENTATION

This section provides a detailed description of the experimental methods used, including how data was collected and organized, and the creation and training of the processing pipeline and machine learning models. While the preceding methodology section elucidated the theoretical underpinnings of each phase, this segment focuses on the specific configurations, parameter selections, utilized tools, and decisions implemented throughout the project.

A. Measurement Setup

All of the experiments took place inside at the Frankfurt University of Applied Sciences. The room was a normal lab with hard walls and a flat floor. The temperature of the air at the time of recording was about 20°C, which was used in the method to figure out the speed of sound.

The ultrasonic sensor was placed horizontally on the edge of a flat table surface and connected to the Red Pitaya measurement board. The sensor was pointed along the length of the table so that the ultrasonic beam went straight ahead, parallel to the surface. Then, a flat object was put in front of the sensor at different starting distances and pushed away by hand in the right direction.

To record at medium speed, the object was pushed gently and steadily so that it moved away at a steady pace. To record at high speeds, the object was pushed quickly and firmly so that it moved away quickly. When recording diagonally, the object was pushed at an angle instead of straight ahead. This meant that the sensor was only following part of the motion instead of the whole movement. In all cases, the object was allowed to come to rest on its own after being pushed, and the sensor kept recording for a few seconds after the movement stopped to make sure the final reading was accurate.

Fig. 1 shows the whole experimental setup, including the ultrasonic sensor mounted on the Red Pitaya board and placed on the measurement surface.

B. Data Format

The sensor data was captured and exported using the measurement software made at Frankfurt University of Applied Sciences. The recordings were all saved as CSV files, with each row representing one measurement frame taken at a rate of 20 Hz. There are two parts to each row: a header section and the echo waveform data.

The first 17 columns (1-based indexing) have header information that tells how the sensor is set up for that frame. Column 7 is the most important header field because it stores the echo sampling frequency of 1,953,125 Hz. This value is needed to change echo sample positions into real distances. The ADC data length, ADC resolution, measurement delay, and a distance value sent by the sensor are other header fields. Most of these extra fields are not used directly in the processing pipeline; they are kept in the file for completeness.

The raw echo waveform is in the columns after column 18. After each ultrasonic pulse was sent, the sensor captured this actual analog-to-digital converted signal. The waveform is made up of thousands of amplitude samples. The strongest negative dip in the waveform shows where the echo from the object was reflected. Table II gives a full explanation of the header fields.

TABLE II
DESCRIPTION OF THE HEADERS IN THE MEASUREMENT FILE

Col	Header Description	Value	Remarks
1	Header Length	68	
2	ADC Data Length	50000	
3	FFT Data Length	0	
4	Feature Data Length	1	
5	Data Type	512	
6	X Interval	1	
7	Sampling Frequency (Hz)	1953125	Echo sampling rate
8	ADC Resolution	14	
9	Ambient Temperature	15	
10	Measurement Delay	2622	
11	Distance	varies	Sensor-reported
12–17	Reserved / Auxiliary	varies	
18+	Echo Waveform Data	varies	Raw ADC samples

The below Fig. 10 shows a screenshot of the raw CSV data as it appears when opened, illustrating the header columns on the left and the echo waveform values on the right.

```

68 50000 0 1 512 1 1953125 14 15 2622 0.71989 0 0 2 2 0 1886.63 -102 -102 -101 -101 -99
68 50000 0 1 512 1 1953125 14 15 2622 0.72312 0 0 2 2 0 1897.96 -107 -112 -99 -99 -90
68 50000 0 1 512 1 1953125 14 15 2622 0.72312 0 0 2 2 0 4127.76 -108 -105 -113 -125 -123
68 50000 0 1 512 1 1953125 14 15 2622 0.72326 0 0 2 2 0 4186.56 -100 -95 -101 -105 -103
68 50000 0 1 512 1 1953125 14 15 2622 0.71797 0 0 2 2 0 4585.67 -96 -95 -98 -99 -102
68 50000 0 1 512 1 1953125 14 15 2622 0.71883 0 0 2 2 0 4608.13 -115 -111 -114 -118 -114
68 50000 0 1 512 1 1953125 14 15 2622 0.72781 0 0 2 2 0 4682.1 -110 -109 -112 -113 -112
68 50000 0 1 512 1 1953125 14 15 2622 0.72783 0 0 2 2 0 4889.25 -106 -110 -110 -111 -104
68 50000 0 1 512 1 1953125 14 15 2622 0.78297 0 0 2 2 0 4942.7 -117 -115 -113 -110 -108
68 50000 0 1 512 1 1953125 14 15 2622 0.78627 0 0 2 2 0 5044.42 -93 -92 -98 -101 -100
68 50000 0 1 512 1 1953125 14 15 2622 0.86613 0 0 2 2 0 5331.07 -101 -102 -99 -97 -97
68 50000 0 1 512 1 1953125 14 15 2622 0.24441 0 0 2 2 0 5174.83 -97 -102 -104 -105 -108
68 50000 0 1 512 1 1953125 14 15 2622 0.8117 0 0 2 2 0 5176.96 -91 -89 -98 -99 -95
68 50000 0 1 512 1 1953125 14 15 2622 0.72701 0 0 2 2 0 5478.98 -100 -94 -94 -97 -80
68 50000 0 1 512 1 1953125 14 15 2622 0.26667 0 0 2 2 0 5551.79 -100 -101 -105 -103 -106
68 50000 0 1 512 1 1953125 14 15 2622 1.71325 0 0 2 2 0 5653.93 -105 -104 -101 -104 -100
68 50000 0 1 512 1 1953125 14 15 2622 1.71814 0 0 2 2 0 5755.98 -97 -103 -109 -109 -103
68 50000 0 1 512 1 1953125 14 15 2622 0.30564 0 0 2 2 0 5859.13 -99 -103 -108 -105 -108
68 50000 0 1 512 1 1953125 14 15 2622 0.30156 0 0 2 2 0 5863.08 -101 -99 -99 -102 -102
68 50000 0 1 512 1 1953125 14 15 2622 0.31094 0 0 2 2 0 6086.2 -117 -115 -114 -101 -102
68 50000 0 1 512 1 1953125 14 15 2622 0.31695 0 0 2 2 0 6164.45 -96 -93 -97 -102 -107
68 50000 0 1 512 1 1953125 14 15 2622 0.32467 0 0 2 2 0 6166.86 -103 -101 -105 -104 -105
68 50000 0 1 512 1 1953125 14 15 2622 0.34854 0 0 2 2 0 6337.92 -104 -100 -102 -104 -105

```

Fig. 10. Raw CSV Data

C. Dataset Collection

Initially, 26 CSV files were gathered from several recording sessions. Each file shows one experimental trial in which an object was either kept still or pushed away from the sensor in different situations. The recordings are different in how far they start, how fast they move, and which way they push.

25 of the 26 files were found to be useful for the classification task. One file was left out because it did not fit into any of the three motion categories that were set up. Table III shows that the 25 usable files were spread out over three motion classes.

TABLE III
DESCRIPTION OF THE DATASET

Motion Type	Recordings	Use
Medium	7	Classification
Fast	10	Classification
Diagonal	8	Classification
Excluded	1	Not matching any class

The medium recordings have files of the object being pushed at a slow, steady pace. Starting distances were anywhere from 1 cm to about 71 cm away from the sensor. The fast recordings show times when the object was pushed quickly. Some of these began very close to the sensor, at a distance of 1 to 8 cm, while others began at a moderate distance of 22 to 86 cm. The diagonal recordings were made by pushing the object at an angle to the sensor's beam axis. These recordings usually show starting distances that are higher, between 1.0 and 1.9 m, because the sensor measures the path at an angle instead of the direct distance.

Each recording was about 10 seconds long and made about 200 frames of echo data at a frame rate of 20 Hz. Some recordings were longer, with up to 600 frames for longer observation periods.

Some of the recordings showed the wall-read problem that was described in the methodology. This happened when the object left the sensor's detection range and the sensor locked onto a background surface instead. A stationary recording was also made, in which nothing was moved. This recording was used as a baseline during the development phase. The automated filtering logic in the processing pipeline found and removed these problematic and special-case recordings from the final classification dataset.

D. Data Processing Pipeline

Python was used to build the whole processing pipeline, which used a number of standard scientific computing and machine learning libraries. Below are the main libraries that were used:

- NumPy: Array operations and numerical computations
- Pandas: Reading CSV files and organizing data into dataframes
- SciPy (medfilt): Median filtering for noise reduction
- scikit-learn: Machine learning models, cross-validation, scaling, and evaluation metrics
- Matplotlib: Plotting confusion matrices and distance-time graphs

The pipeline runs in a fully automated manner. Given a folder containing CSV files, it performs the following sequence of operations for each file:

- The CSV file is read, and column 7 (index 6 in zero-based Python indexing) is used to get the echo sampling frequency.
- The raw echo waveform starts at column 18 (index 17 in zero-based indexing). The peak index is found by locating the sample with the lowest amplitude in each frame.
- The time-of-flight formula is used to turn the peak index into a real distance. The speed of sound is calculated at 20°C.
- A median filter with a kernel size of 5 is used to smooth out the distance signal.
- The sliding window algorithm finds the stable start and end positions.

- The regression speed is found by dividing the displacement by the duration.
- The MAE and RMSE are calculated by comparing the regression line to the actual filtered distance during the motion period.
- 13 features are extracted from the processed signal.
- The file is automatically labeled as Medium, Fast, or Diagonal based on keywords in the filename. Files containing words like "stationary," "human," or "slow" are automatically excluded.

The overall system pipeline from raw CSV input to final classification output is illustrated in Fig. 11.

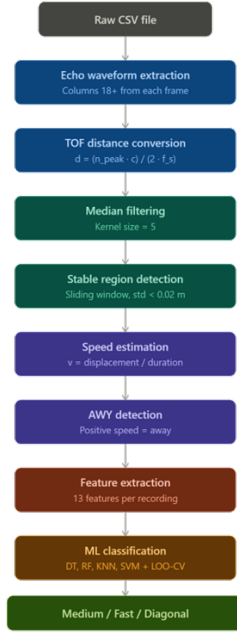


Fig. 11. System Flowchart

E. Model Building and Training

All four machine learning models were implemented using the `scikit-learn` library in Python. `StandardScaler` was used to normalize all 13 features before training, changing the mean of each feature to 0 and the standard deviation to 1. This normalization was done separately for each fold of the Leave-One-Out Cross-Validation so that no information from the test sample got into the training data.

The following settings were used to initialize the four models:

TABLE IV
MODEL PARAMETERS USED FOR CLASSIFICATION

Model	Parameters
Decision Tree	<code>max_depth=3, random_state=42</code>
Random Forest	<code>n_estimators=100, max_depth=3, random_state=42</code>
KNN	<code>n_neighbors=3</code>
SVM	<code>kernel='rbf', random_state=42</code>

Decision Tree: To keep the Decision Tree from overfitting, it was set to a maximum depth of 3. The parameter `random_state=42` makes sure that the results can be repeated. It learned from the scaled training split of each LOO-CV round and was then used to guess the class of the test sample that was not used.

```

model = DecisionTreeClassifier(
    max_depth=3, random_state=42)
Xtr_s = scaler.fit_transform(Xtr)
Xte_s = scaler.transform(Xte)
model.fit(Xtr_s, ytr)
yp.append(model.predict(Xte_s)[0])

```

Random Forest: The Random Forest was made up of 100 decision trees, each with a maximum depth of 3. At each split, a random group of features is looked at. This makes the trees different and helps prevent overfitting. All 100 trees vote on the final prediction, and the one with the most votes wins.

```

model = RandomForestClassifier(
    n_estimators=100, max_depth=3,
    random_state=42)
Xtr_s = scaler.fit_transform(Xtr)
Xte_s = scaler.transform(Xte)
model.fit(Xtr_s, ytr)
yp.append(model.predict(Xte_s)[0])

```

KNN ($k=3$): KNN keeps all 24 training samples from each round and uses *Euclidean distance* to find the 3 closest neighbors of the test sample. In this case, feature scaling is very important because unscaled features with big ranges would take over the distance calculation.

```

model = KNeighborsClassifier(
    n_neighbors=3)
Xtr_s = scaler.fit_transform(Xtr)
Xte_s = scaler.transform(Xte)
model.fit(Xtr_s, ytr)
yp.append(model.predict(Xte_s)[0])

```

SVM: The SVM was configured with an RBF kernel to handle non-linear decision boundaries between the three motion classes. Feature scaling is especially important for SVM because its optimization depends on distances in the feature space.

```

model = SVC(
    kernel='rbf', random_state=42)
Xtr_s = scaler.fit_transform(Xtr)
Xte_s = scaler.transform(Xte)
model.fit(Xtr_s, ytr)
yp.append(model.predict(Xte_s)[0])

```

The `random_state=42` parameter was set for the Decision Tree, Random Forest, and SVM models to ensure that results are the same every time they are run. The Random Forest had 100 trees, and each tree could have a maximum depth of 3 levels. The default *Euclidean distance* metric was used for KNN, with k set to 3 neighbors. The RBF kernel was chosen for SVM so that the decision boundaries could be non-linear.

The Leave-One-Out Cross-Validation (LOO-CV) procedure was used for both training and evaluation. One of the 25 recordings was held out as the test sample, and the remaining 24 recordings were used for training. The `StandardScaler` was trained on the 24 training samples and then used on both the training set and the one test sample. The model was then trained on the scaled training data and used to predict the class of the held-out test sample. After all 25 rounds, the predictions were collected and used to calculate the accuracy, confusion matrix, precision, recall, and $F1$ -score.

Fig. 12 shows the complete model training loop used across all four classifiers within the LOO-CV procedure.

```
models = {
    'Decision Tree': DecisionTreeClassifier(random_state=42, max_depth=3),
    'Random Forest': RandomForestClassifier(random_state=42, n_estimators=100, max_depth=3),
    'KNN (k=3)': KNeighborsClassifier(n_neighbors=3),
    'SVM': SVC(kernel='rbf', random_state=42),
}

print(f'\n(={*60})')
print(" MODEL COMPARISON (Leave-One-Out Cross-Validation)")
print(f'\n(={*60})\n')

best_acc = 0
best_model = ""
all_preds = {}

for mname, model in models.items():
    yt = []
    yp = []

    for tri, tei in loo.split(X):
        Xtr, Xte = X.iloc[tri], X.iloc[tei]
        ytr, yte = y[tri], y[tei]
        Xtr_s = scaler.fit_transform(Xtr)
        Xte_s = scaler.transform(Xte)
        model.fit(Xtr_s, ytr)
        yp.append(model.predict(Xte_s)[0])
        yt.append(yte[0])

    acc = accuracy_score(yt, yp)
    all_preds[mname] = (yt, yp, acc)
    n_correct = sum(a == b for a, b in zip(yt, yp))

    if acc > best_acc:
        best_acc = acc
        best_model = mname
```

Fig. 12. Code snippet illustrating model initialization and training within the LOO-CV loop.

A Random Forest model was also trained on the entire scaled dataset to determine the importance of each feature. These scores indicate how much each of the 13 features contributed to the classification decisions of the 100 trees in the forest. The features were then ranked from most to least important to identify which ones were most effective at distinguishing between the three types of motion.

F. Confusion Matrix Generation

After the LOO-CV evaluation was done on all four models, a confusion matrix was generated for each one. The `confusion_matrix` function from `scikit-learn` was used to compute the matrix. It takes a list of actual class labels and predicted class labels and produces a 3×3 matrix.

To make the results easier to interpret visually, the four confusion matrices were plotted next to each other in a 2×2 grid using `Matplotlib`. A blue color map was used to color-code each cell in the matrix, with darker shades indicating

higher counts. The actual count was displayed as a number in the center of each cell. The title of each subplot includes the model name and its overall accuracy. This combined confusion matrix plot is shown in the Results section as Fig. 15.

The `classification_report` function from `scikit-learn` was used to generate the classification report, which shows the precision, recall, and $F1$ -score for each class and each model. Along with the confusion matrices, these detailed metrics were printed in the console output to provide a comprehensive view of each model's strengths and weaknesses across all three motion classes.

IV. RESULTS AND DISCUSSION

This section presents the results obtained from the speed estimation pipeline and the machine learning classification models. The results are organized into several parts: speed estimation and AWY detection performance, classification model comparison, confusion matrix analysis, detailed per-class metrics, feature importance analysis, and per-file prediction analysis. All results shown in the figures were generated directly from the Python pipeline output.

A. Speed Estimation and AWY Detection

The linear regression model $d(t) = d_{\text{start}} + v \cdot (t - t_{\text{start}})$ was applied to all 25 recordings to estimate the speed of each object movement. For each file, the regression speed, the MAE, and the RMSE of the regression fit were computed. Fig. 13 shows the complete speed estimation output for all 25 recordings.

Ultrasonic Sensor: Speed & Motion Dashboard														
Raw Extracted Features (Per-File)														
Total Files	Total Features	Overall Mean RMSE			Overall Mean MAE			AWY Detected						
25	13	0.4081 m			0.4806 m			25/25						
Filename	Label	start_distance	end_distance	displacement	duration	regression_speed	awy_detected	reg_rmse	reg_mae	rmse_displacement	displacement_ratio	mean_velocity	std_velocity	mean_velocity_percent
d1a 1.csv	Diagonal	1.9897	2.232	0.2424	3.48	0.204		0.295	0.3373	0	0	1.143105	204.4712	90.079
d1a 10.csv	Diagonal	1.8977	2.223	0.3443	2.38	0.4029		0.3484	0.3787	0	0	1.184108	205.4127	94.633
d1a 11.csv	Diagonal	1.5996	2.1158	0.5162	1.4	0.3718		0.2247	0.2486	0	0	1.063315	247.8515	83.348
d1a 13.csv	Diagonal	1.5983	2.222	0.6239	1.78	0.3576		0.3253	0.3086	0	0	1.276175	204.1557	103.287
d1a 15.csv	Diagonal	1.6019	2.2082	0.6063	4.05	0.4097		0.3458	0.382	0	0	1.088102	270.8478	89.9025
d1a0405060708_200.csv	Diagonal	0.0047	1.9064	1.9067	1.1	1.7079		0.5059	0.8012	32	0.18	988.62	290.132	72.1234
d1a0405060708_200.csv	Diagonal	0.0063	1.9016	1.9016	1.1	1.6465		0.5022	0.804	31	0.18	987.259	290.8375	71.7127
d1a0405060708_200.csv	Diagonal	0.0054	1.9141	0.9086	1.1	0.4039		0.309	0.367	9	0.06	1.378105	260.19	93.891
fast 1.csv	Fast	0.0002	0.0168	0.0166	0.78	0.1115		0.0014	0.0007	0	0	2.145125	701.4568	101.091
fast 13.csv	Fast	0.1201	1.9861	1.866	2.76	0.5313		0.2202	0.3877	2	0.01	1.061717	407.6158	87.5261

Fig. 13. Streamlit dashboard Tab 1 showing all 25 recordings with 13 extracted features, AWY detection status, MAE, and RMSE for every file.

The above figure shows the speed estimation summary grouped by motion class. The speed values show a clear separation between the three motion types. Medium recordings had the lowest speeds, ranging from 0.025 to 0.461 m/s with a mean of 0.197 m/s. Fast recordings had the highest speeds, reaching up to 2.322 m/s with a mean of 1.022 m/s. Diagonal recordings fell in between, with a mean speed of 0.617 m/s, though the range overlapped with both the fast and medium categories.

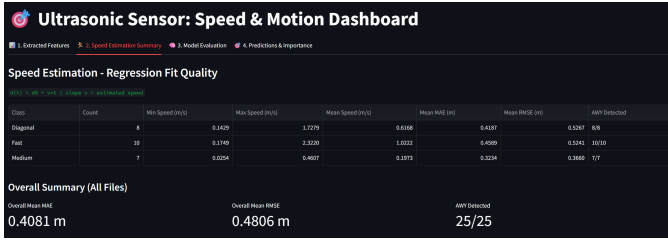


Fig. 14. Streamlit dashboard Tab 2 showing regression fit quality grouped by motion class, with min, max, and mean speeds alongside mean MAE, mean RMSE, and AWY detection rate for Diagonal, Fast, and Medium classes.

The MAE and RMSE values indicate how well the linear regression model fit the actual sensor data during the motion period. Medium recordings had the lowest error values (MAE = 0.323 m, RMSE = 0.366 m), suggesting that the signal during medium-speed motion followed the expected linear path more closely. Fast and diagonal recordings had higher error values, which is expected because faster motion causes more echo dropouts and signal disruptions, making the actual distance signal deviate more from the ideal straight line. The overall mean MAE across all 25 files was 0.408 m and the mean RMSE was 0.481 m.

AWY movement was successfully detected in all 25 recordings. Every file produced a positive regression speed, confirming that the object moved away from the sensor in every trial. This gives an AWY detection rate of 100% across the entire dataset.

B. Classification Model Comparison

Four machine learning classifiers were trained and evaluated using Leave-One-Out Cross-Validation on the 25 recordings. The SVM classifier achieved the highest accuracy of 88.0%, correctly classifying 22 out of 25 recordings. Random Forest achieved 80.0%, while Decision Tree and KNN both achieved 76.0%.

The SVM performed particularly well because of its ability to focus on the boundary region between overlapping classes. With the RBF kernel, SVM was able to create flexible, non-linear decision boundaries that could separate the three motion types even where their feature values overlapped. The Decision Tree performed worst because a single tree with a maximum depth of 3 has limited capacity to capture the complex relationships between the 13 features.

TABLE V
PRECISION, RECALL, AND F1-SCORE FOR ALL MODELS
(LEAVE-ONE-OUT CROSS-VALIDATION)

Model	Class	Precision	Recall	F1-Score	Support
Decision Tree (76.0%)	Diagonal	0.875	0.875	0.875	8
	Fast	0.700	0.700	0.700	10
	Medium	0.714	0.714	0.714	7
	Accuracy			0.760	25
	Macro Avg	0.763	0.763	0.763	25
	Weighted Avg	0.760	0.760	0.760	25
Random Forest (80.0%)	Diagonal	0.778	0.875	0.824	8
	Fast	0.800	0.800	0.800	10
	Medium	0.833	0.714	0.769	7
	Accuracy			0.800	25
	Macro Avg	0.804	0.796	0.798	25
	Weighted Avg	0.802	0.800	0.799	25
KNN (k=3) (76.0%)	Diagonal	0.714	0.625	0.667	8
	Fast	0.727	0.800	0.762	10
	Medium	0.857	0.857	0.857	7
	Accuracy			0.760	25
	Macro Avg	0.766	0.761	0.762	25
	Weighted Avg	0.759	0.760	0.758	25
SVM (88.0%)	Diagonal	1.000	0.875	0.933	8
	Fast	0.769	1.000	0.870	10
	Medium	1.000	0.714	0.833	7
	Accuracy			0.880	25
	Macro Avg	0.923	0.863	0.879	25
	Weighted Avg	0.908	0.880	0.880	25

C. Confusion Matrix Analysis

The confusion matrices for all four models are shown below. Each matrix is a 3×3 table where the rows represent the actual class and the columns represent the predicted class. The diagonal cells show correct predictions, and the off-diagonal cells show misclassifications. Darker blue shading indicates higher counts.

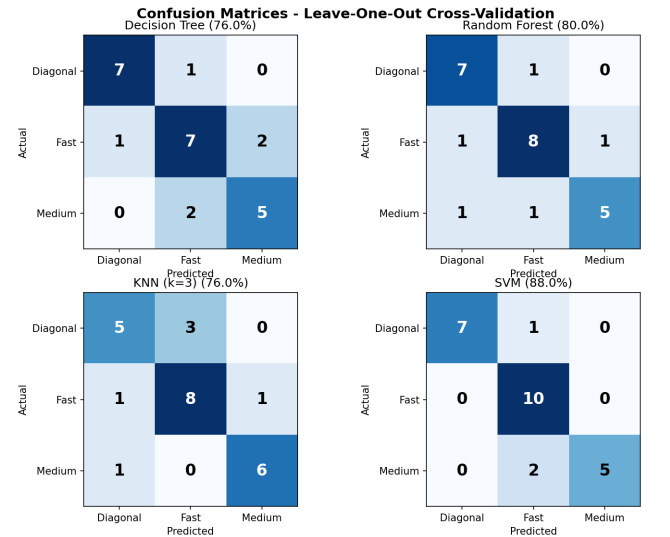


Fig. 15. Confusion matrices for all four classifiers under Leave-One-Out Cross-Validation: Decision Tree (76%), Random Forest (80%), KNN (76%), and SVM (88%).

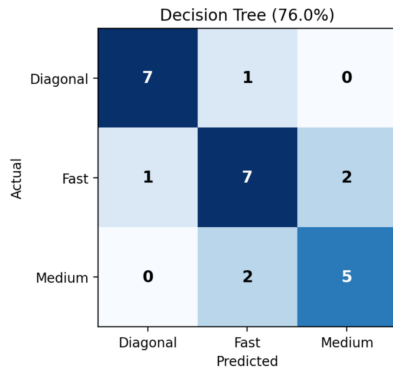


Fig. 16. Confusion matrix for the Decision Tree classifier (76.0% accuracy).

Decision Tree (76.0%): The Decision Tree put 19 out of 25 recordings in the right category. For the Diagonal class, 7 out of 8 recordings were correctly identified, but 1 was incorrectly labeled as Fast. Only 7 out of 10 recordings were correctly classified as Fast; 1 was wrongly classified as Diagonal and 2 were wrongly classified as Medium. This is the most misclassifications for the Fast class across all models. This is because a single tree with a maximum depth of 3 cannot capture the boundary between Fast and Medium very well. For the Medium class, 5 out of 7 recordings were correctly classified, but 2 were incorrectly classified as Fast.

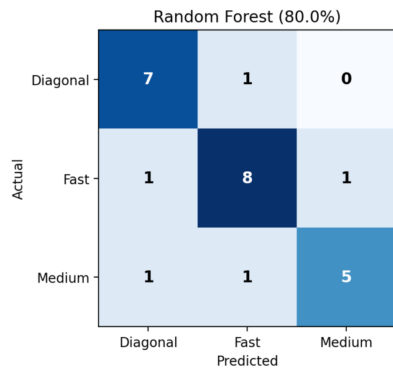


Fig. 17. Confusion matrix for the Random Forest classifier (80.0% accuracy).

Random Forest (80.0%): The Random Forest got 20 out of 25 recordings right, which is one more than the Decision Tree. 7 out of 8 recordings in the Diagonal class were correctly identified, but 1 was wrongly labeled as Fast. For the Fast class, 8 out of 10 recordings were correctly classified. 1 was incorrectly classified as Diagonal and another as Medium. For the Medium class, 5 out of 7 recordings were correctly classified. 1 was incorrectly classified as Diagonal and 1 as Fast. Random Forest is an ensemble method that combines the votes of 100 trees. This reduces the variance compared to a single Decision Tree and leads to a more balanced error distribution across all three classes.

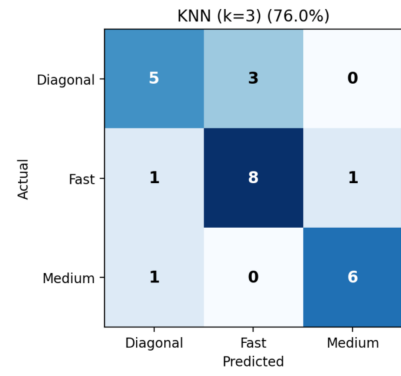


Fig. 18. Confusion matrix for the KNN ($k=3$) classifier (76.0% accuracy).

KNN ($k=3$) (76.0%): KNN also got 76.0% accuracy, putting 19 out of 25 recordings in the right category. The Diagonal class did the worst across all four models, with only 5 out of 8 recordings being correctly identified. 3 of these were incorrectly classified as Fast. This means that some Diagonal recordings have feature vectors that are geometrically close to Fast recordings in the 13-dimensional space. When $k = 3$, the nearest neighbors are often Fast samples. 8 out of 10 recordings in the Fast class were correctly classified, but 1 was incorrectly classified as Diagonal and another as Medium. The KNN model did the best for the Medium class, correctly classifying 6 out of 7 cases. Only 1 case was misclassified as Diagonal. This is because Medium recordings tend to group together in the feature space because they always have low speeds and high dropout counts, which makes them easy for distance-based classifiers to identify.

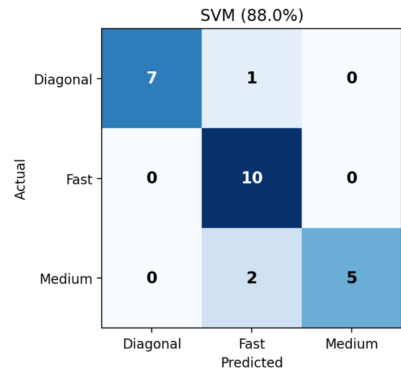


Fig. 19. Confusion matrix for the SVM classifier (88.0% accuracy).

SVM (88.0%): SVM had the best overall performance, correctly classifying 22 out of 25 recordings with an accuracy of 88.0%. 7 out of 8 recordings in the Diagonal class were correctly identified. There was only one diagonal file that was incorrectly labeled as Fast: `diagonalobjectsmallldist_200.csv`. This recording started at a distance of 1.03 m, which is much lower than the other diagonal files, which usually start at 1.59 to 1.86 m. This makes its features more like those of Fast recordings. SVM got

a perfect score of 10 out of 10 for the Fast class, which was the only perfect score for any model in this study. 5 out of 7 recordings were correctly classified for the Medium class. The two wrong classifications were medium 1.csv with a speed of 0.262 m/s and medium 10 distance 10cm.csv with a speed of 0.461 m/s. Both of these speeds are in the Fast speed range.

One thing that all four models have in common is that confusion mostly happens between the Fast and Medium classes. It is easy to tell the difference between diagonal recordings because they have very different distance ranges. For example, diagonal recordings start at 1.0 to 1.9 m, while the other classes start at 0.01 to 0.86 m.

D. Precision, Recall, and F1-Score

The detailed precision, recall, and $F1$ -score for each class under each model were generated using the `classification_report` function from `scikit-learn`. The complete output for all four models is shown in Table VI.

TABLE VI
PRECISION, RECALL, AND $F1$ -SCORE FOR ALL MODELS

Model	Class	Precision	Recall	$F1$ -Score
Decision Tree	Diagonal	0.875	0.875	0.875
	Fast	0.700	0.700	0.700
	Medium	0.714	0.714	0.714
Random Forest	Diagonal	0.778	0.875	0.824
	Fast	0.800	0.800	0.800
	Medium	0.833	0.714	0.769
KNN ($k = 3$)	Diagonal	0.714	0.625	0.667
	Fast	0.727	0.800	0.762
	Medium	0.857	0.857	0.857
SVM	Diagonal	1.000	0.875	0.933
	Fast	0.769	1.000	0.870
	Medium	1.000	0.714	0.833

The SVM model achieved the highest macro-average $F1$ -score of 0.879, followed by Random Forest at 0.798, Decision Tree at 0.763, and KNN at 0.762.

E. Feature Importance

The feature importance ranking from the Random Forest model is shown below. These scores indicate how much each feature contributed to the classification decisions across all 100 trees in the ensemble.

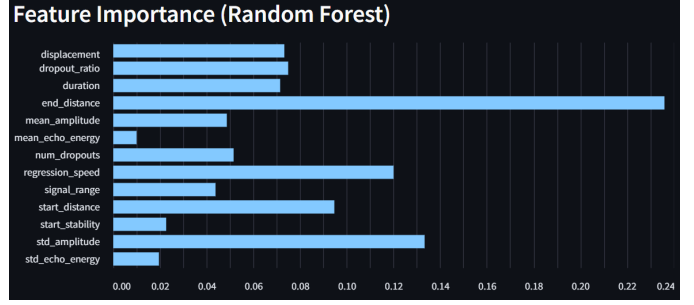


Fig. 20. Feature importance scores from the Random Forest model, ranked from most to least important across all 13 extracted features.

The most important feature by a significant margin is End Distance, with an importance score of 0.236. This makes sense because the end distance is the primary feature that separates diagonal recordings (which end at 2.1 to 2.2 m) from medium and fast recordings (which end at 0.5 to 1.6 m). The Random Forest likely uses this feature at the top of many of its trees to immediately split off the diagonal class.

The second most important feature is Std Amplitude (0.133), which captures the variation in echo strength across frames. This variation tends to be higher during fast motion due to rapid changes in the reflected signal, making it useful for separating fast from medium recordings.

Regression Speed ranks third (0.120), which is expected since speed is the most direct indicator of the motion type. Start Distance (0.095) is also important because diagonal recordings consistently start at greater distances than the other two classes.

F. Per-File Prediction Analysis

Out of 25 recordings, 22 were correctly classified and 3 were misclassified by the best model (SVM). All three misclassifications involved files being predicted as Fast when they belonged to a different class. The misclassified files share a common trait: their feature values fall in the overlapping region between their actual class and the Fast class.

Per-File Predictions (SVM)			
Filename	Actual	Predicted	Result
dia 1.csv	Diagonal	Diagonal	✓ OK
dia 10.csv	Diagonal	Diagonal	✓ OK
dia 2.csv	Diagonal	Diagonal	✓ OK
dia 4.csv	Diagonal	Diagonal	✓ OK
dia 5.csv	Diagonal	Diagonal	✓ OK
diagonalobjectleft_200.csv	Diagonal	Diagonal	✓ OK
diagonalobjectrange_200.csv	Diagonal	Diagonal	✓ OK
diagonalobjectsmalldist_200.csv	Diagonal	Fast	✗ WRONG
fast 10.csv	Fast	Fast	✓ OK
fast 3.csv	Fast	Fast	✓ OK

Fig. 21. Streamlit dashboard Tab 4 showing per-file SVM predictions with correct and incorrect classifications on the left, and Random Forest feature importance bar chart on the right with end_distance ranked as the most important feature.

G. Summary of Key Results

The key findings from the results are summarized below:

- AWY movement was detected with 100% accuracy across all 25 recordings.
- The SVM classifier achieved the highest classification accuracy of 88.0% with a macro-average $F1$ -score of 0.879.
- The overall regression fit quality showed a mean MAE of 0.408 m and mean RMSE of 0.481 m.
- Medium recordings had the best regression fit (MAE = 0.323 m) due to fewer echo dropouts.
- End Distance was the most important feature for classification, primarily because it separates the Diagonal class from the other two.
- Classification errors occurred mainly at the boundary between the Fast and Medium classes, where feature values overlap.
- The small dataset size of 25 samples is a limitation, and collecting more recordings would likely improve classification accuracy further.

V. CONCLUSION

This project created and tested a system that uses machine learning to find objects moving away from a single horizontally mounted ultrasonic sensor and classify the type of motion they are making. The system takes raw echo waveforms from the SRF02 sensor, uses the time-of-flight principle to turn them into distance, applies median filtering, uses a linear regression model to estimate speed, and uses extracted signal features to put the motion into three groups.

There were 25 experimental recordings, and the system was able to find AWY movement in all of them, which is a 100% success rate. The SVM model was the best of the four classifiers tested, with an accuracy of 88.0% and a macro-average $F1$ -score of 0.879. The results show that a single cheap ultrasonic sensor and a lightweight signal processing and machine learning pipeline can be used to reliably detect movement away from the sensor and classify motion based on speed.

The primary limitations of this study are the limited dataset comprising 25 recordings, which constrains the classifiers' generalization ability, and the echo dropout issue intrinsic to the sensor, which compromises the quality of regression fitting during rapid motion. Classification mistakes were mostly found at the line between the Fast and Medium classes, where feature values overlap.

In the future, researchers could add more recordings at different speeds and distances to the dataset, look into Doppler-based speed estimation for more direct velocity measurement, and use the system in real time to keep an eye on motion all the time.

VI. ACKNOWLEDGMENT

We want to thank Prof. Dr. Andreas Pech from the bottom of our hearts for giving us the chance to work on this project with him and for guiding us in the right direction all along the way. The "Machine Learning" lectures were very helpful because they gave us the background knowledge and motivation we needed to do this work. We also want to thank Prof. Julian Umansky for his important help in giving us all the tools and sensor equipment we needed to finish this project successfully. Lastly, we want to thank the authors of all the references and works cited in this report. Without their work, this report would not have been possible.

REFERENCES

- [1] N. Thong-un and W. Wongsaroj, "A Novel Ultrasonic Method for Measuring the Position and Velocity of Moving Objects in 3D Space," *Advances in Technology Innovation*, vol. 7, no. 2, pp. 77–91, 2022.
- [2] L. Alonso, J. P. Riera, and M. A. González, "Ultrasonic Sensors in Urban Traffic Driving-Aid Systems," *Sensors*, vol. 11, no. 1, pp. 661–676, 2011.
- [3] A. L. Bowler et al., "Ultrasonic sensing and machine learning for industrial process monitoring," *Ultrasonics*, vol. 120, 2022.
- [4] "SRF02 Ultrasonic range finder," *Robot Electronics*. [Online]. Available: <https://www.robot-electronics.co.uk/htm/srf02tech.htm>. [Accessed: Mar. 14, 2026]
- [5] "Red Pitaya STEMLab," Red Pitaya. [Online]. Available: <https://redpitaya.com/stemlab-125-14/>
- [6] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA: Addison-Wesley, 1977.
- [7] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [8] L. Breiman, J. Friedman, C. J. Stone, and R. A. Olshen, *Classification and Regression Trees*. CRC Press, 1984.
- [9] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [10] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inform. Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [11] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [12] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proc. 14th Int. Joint Conf. Artif. Intell.*, 1995, pp. 1137–1143.
- [13] J. Ritari, K. Hyvärinen, S. Koskela, et al., "Genomic prediction of relapse in recipients of allogeneic haematopoietic stem cell transplantation," *Leukemia*, vol. 33, pp. 240–248, 2019. doi: 10.1038/s41375-018-0229-3.
- [14] B. Satria, N. Afrianto, L. Ningsih, P. Sakinah, A. Sidauruk, and L. Mayola, "Comparative analysis of weighted-KNN, random forest, and support vector machine models for beef and pork image classification using machine learning," *JOIV: International Journal on Informatics Visualization*, vol. 9, no. 4, pp. 1677–1687, Jul. 2025. doi: 10.62527/joiv.9.4.3736.
- [15] I. D. Mienye and N. R. Jere, "A survey of decision trees: Concepts, algorithms, and applications," *IEEE Access*, vol. 12, pp. 86716–86727, 2024. doi: 10.1109/ACCESS.2024.3416838.